SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO* (BL4 , BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **TOTAL MARKS: 25**

EXPERIMENTS

Code of Conduct:

I ___DOMA SAKETH-192525018__(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

D.Saketh

Annexure A
Experiments

1. **Implement and compare MD5 and Secure Hash Algorithm (SHA-256) in Python by hashing multiple input messages. Analyze their collision resistance and performance differences.**

AIM: To implement and compare **MD5** and **SHA-256** hashing algorithms in Python using multiple input messages and analyze their hash length, collision resistance, and performance.

Algorithm : MD5 Hashing

1. **Start**
2. Read the input message.
3. Convert the message into bytes using UTF-8 encoding.
4. Initialize the **MD5** hash function.
5. Pass the message bytes to the MD5 algorithm.
6. Generate the **128-bit hash value**.
7. Convert the hash into a **32-character hexadecimal string**.
8. Display the MD5 hash.
9. **Stop**

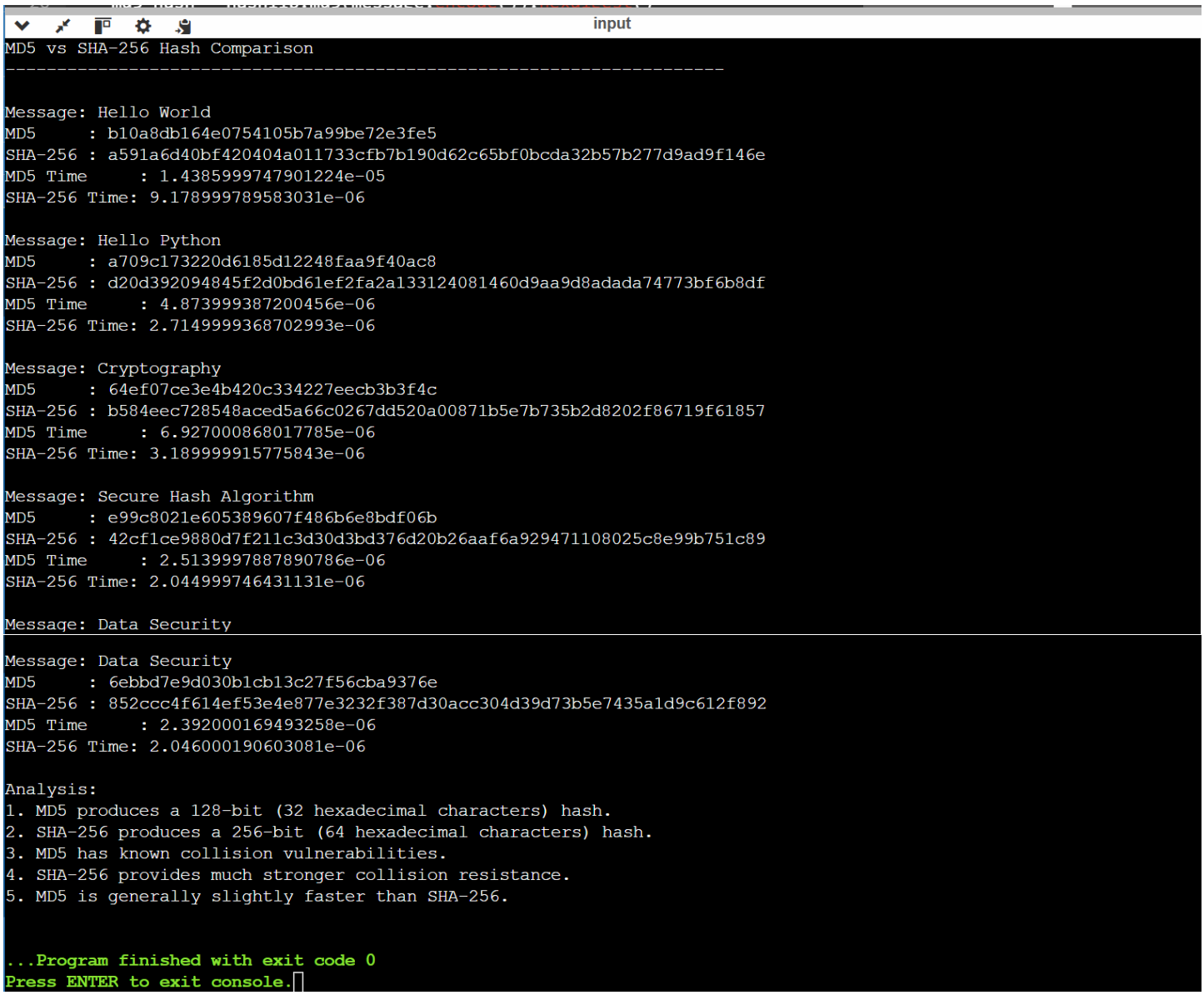
Python Program:

```
import hashlib

import time



# Input messages

messages = [

    "Hello World",

    "Hello Python",

    "Cryptography",

    "Secure Hash Algorithm",

    "Data Security"

]



print("MD5 vs SHA-256 Hash Comparison")

print("-" * 70)



for message in messages:



    # MD5 hashing
```

```
    start = time.perf_counter()
    md5_hash = hashlib.md5(message.encode()).hexdigest()
    md5_time = time.perf_counter() - start

    # SHA-256 hashing
    start = time.perf_counter()
    sha256_hash = hashlib.sha256(message.encode()).hexdigest()
    sha256_time = time.perf_counter() - start

    print("\nMessage:", message)
    print("MD5     :", md5_hash)
    print("SHA-256 :", sha256_hash)
    print("MD5 Time     :", md5_time)
    print("SHA-256 Time:", sha256_time)

print("\nAnalysis:")
print("1. MD5 produces a 128-bit (32 hexadecimal characters) hash.")
print("2. SHA-256 produces a 256-bit (64 hexadecimal characters) hash.")
```

```
print("3. MD5 has known collision vulnerabilities.")

print("4. SHA-256 provides much stronger collision resistance.")

print("5. MD5 is generally slightly faster than SHA-256.")
```

OUTPUT:



Test Cases:

Test Case	Input Message	Expected Result
TC01	Hello World	MD5 and SHA-256 hashes generated successfully
TC02	Hello Python	MD5 and SHA-256 hashes generated successfully
TC03	Cryptography	MD5 and SHA-256 hashes generated successfully
TC04	Secure Hash Algorithm	MD5 and SHA-256 hashes generated successfully
TC05	Data Security	MD5 and SHA-256 hashes generated successfully
TC06	1234567890	MD5 and SHA-256 hashes generated successfully
TC07	"" (empty string)	MD5 and SHA-256 hashes generated successfully
TC08	Hello World!	Hash should differ from Hello World
TC09	hello world	Hash should differ from Hello World
TC10	A	MD5 and SHA-256 hashes generated successfully

5. Result:

MD5 and SHA-256 were successfully implemented and compared. SHA-256 is more secure and collision-resistant than MD5, while MD5 is generally faster. Therefore, SHA-256 is preferred for modern security applications.

2. Write a Python program to generate the MD5 hash value of a given text message. Input: "Hello World"

   Output: 32-character hexadecimal MD5 digest.

Aim

To write a Python program to generate the **MD5 hash value** of the given text message "Hello World".

import hashlib.

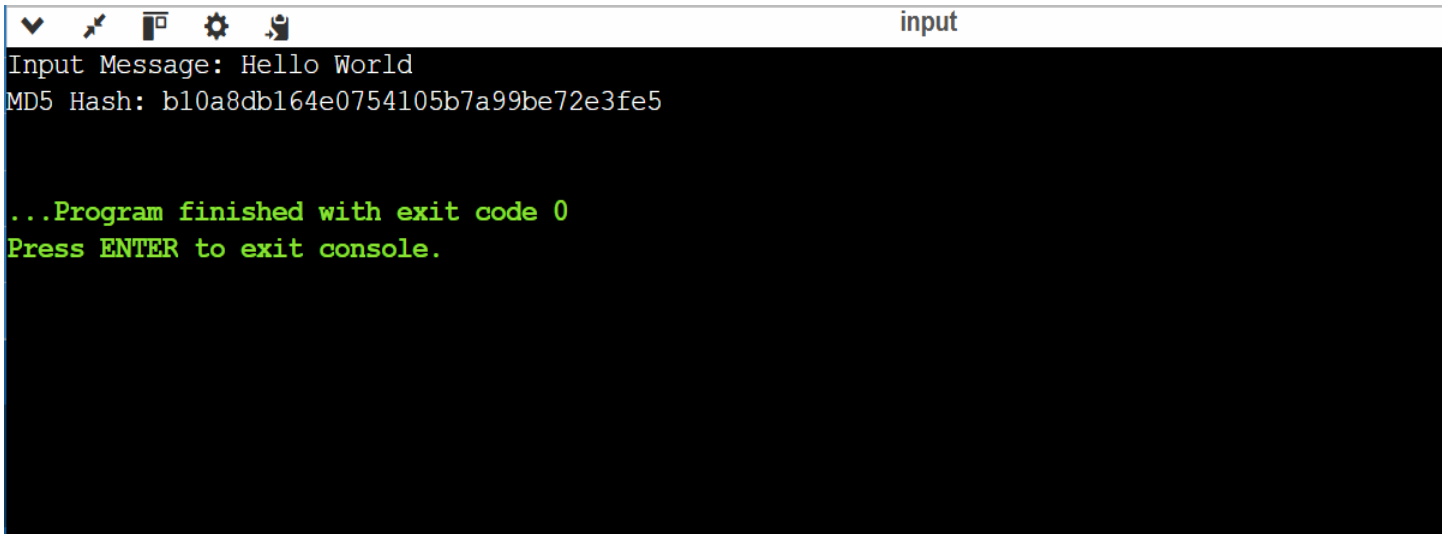
Algorithm : SHA-256 Hashing

1. **Start**
2. Read the input message.
3. Convert the message into bytes using UTF-8 encoding.
4. Initialize the **SHA-256** hash function.
5. Pass the message bytes to the SHA-256 algorithm.
6. Generate the **256-bit hash value**.
7. Convert the hash into a **64-character hexadecimal string**.
8. Display the SHA-256 hash.
9. **Stop**

PROGRAM:

```
# Input message
message = "Hello World"

# Generate MD5 hash
md5_hash = hashlib.md5(message.encode()).hexdigest()

# Display result
print("Input Message:", message)
print("MD5 Hash:", md5_hash)
```

OUTPUT:



TEST CASE:

Test Case	Input	Expected Output
TC01	Hello World	b10a8db164e0754105b7a99be72e3fe5

Result:

The MD5 hash value of the given text **"Hello World"** was successfully generated using Python.

MD5 Digest:
b10a8db164e0754105b7a99be72e3fe5

The output is a **32-character hexadecimal MD5 digest**.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of	15	Insightful analysis	Good	Basic analysis	Poor or

Results		with accurate interpretation and validation of results	analysis with reasonable interpretation	with limited insights	incorrect analysis of results
Documentation	10	Clear, wellstructured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation